High Level Design And Low Level Design—PeerBridge

1. Introduction

1.1 Purpose

This document gives the design of the PeerBridge platform. The High Level Design (HLD) shows the architecture, components and data flow. The Low Level Design (LLD) shows the screens, classes, database, APIs and algorithms needed to code it.

1.2 System Goal

- Let seniors (Year 3-4) create sessions, quizzes and resources for juniors (Year 1-2)
- Let juniors discover and register for sessions, attempt quizzes and rate mentors
- Measure participation using points, badges and two leaderboards
- Use free-tier tools only: React (Vercel), Express (Render), MongoDB Atlas

1.3 Design Decisions Made

These are my design choices where the SRS did not give a value. All can be changed after your review.

No.	Decision	Reason
D1	Role is set by the server from the year of study (1-2 junior, 3-4 senior)	A user cannot pick a role (AUTH-F3)
D2	Layered design: route, controller, service, model	Each of the 3 team members can own modules without conflicts (SYS-7)
D3	Session date and time are stored as one date-time field (startsAt) in UTC and shown in IST	Easy to sort and filter upcoming sessions
D4	Seat booking is one atomic MongoDB update	No double registration and no overbooking, even with two clicks at once (SESS-F6, F7, N1)
D5	One quiz per session and one attempt per student per quiz	Points cannot be farmed by repeating a quiz
D6	Quiz timer is kept on the server (startedAt, deadlineAt). Expired attempts are closed when next accessed	Render free tier sleeps, so a background job is not reliable (QUIZ-N3)
D7	Every point change is saved as a point_transactions record, and the total is kept in users.points	Audit trail and no double award (LEAD-N1, N3)
D8	Point values are proposed and kept in one config file	OI-6 is still open; changing values needs no other code change
D9	Mentor leaderboard lists only mentors with at least 3 ratings	One 5-star rating should not make a top mentor
D10	Rating opens when the session start time has passed. Registration counts as attendance	No attendance tracking in version 1
D11	Resources are links only (http/https). The server never opens them	Broken links cannot crash the page (RES-N2)
D12	Admin panel, AI chatbot and notifications are not designed in this version	OI-1, OI-2, OI-3 are open; treated as future scope

2. High Level Design (HLD)

2.1 System Architecture

PeerBridge uses a 4-layer architecture: Presentation, API, Service and Data. Each layer talks only to the layer below it. Meeting links are only stored and shown; no meeting platform is called.

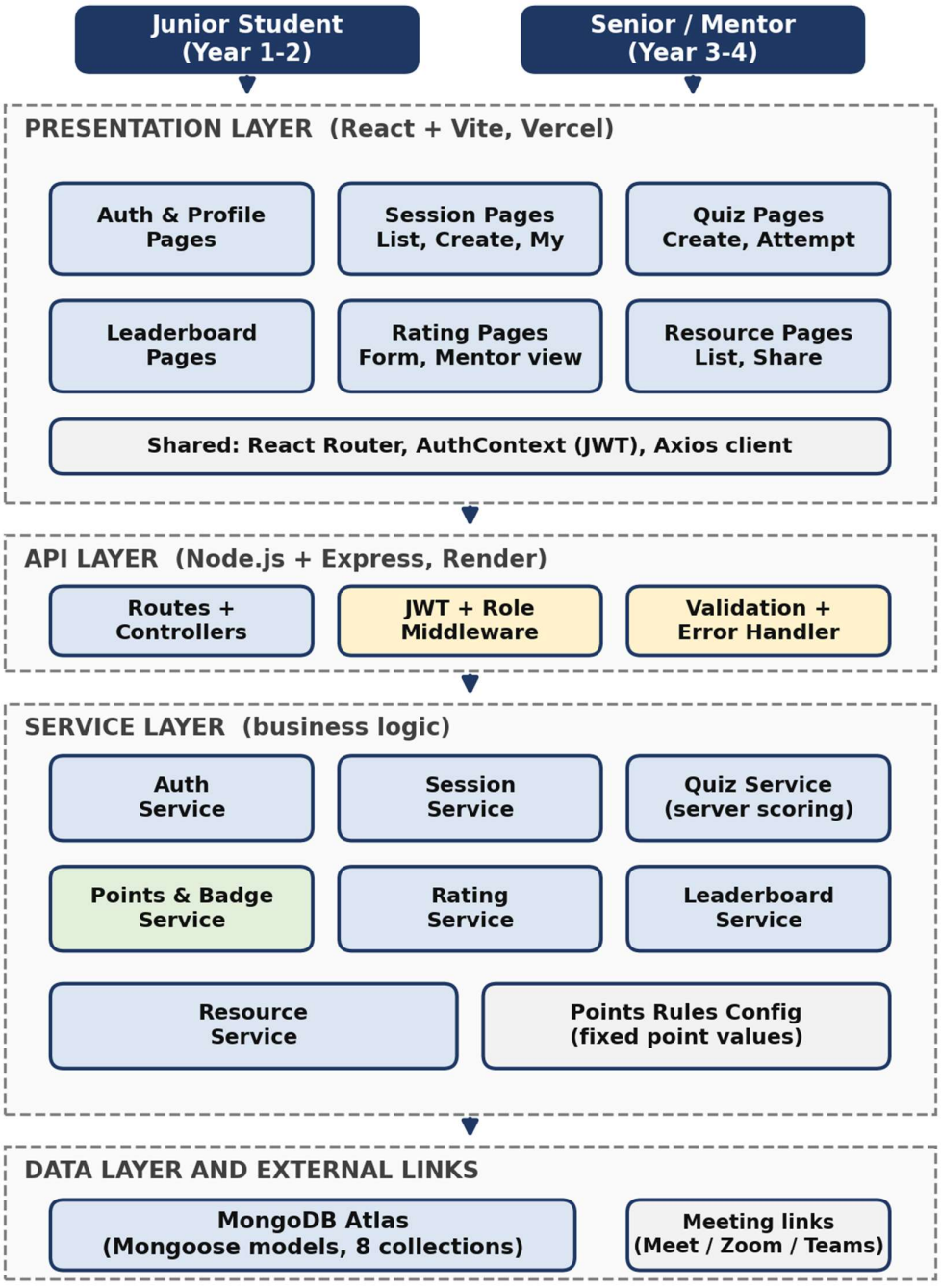


Figure 1: HLD architecture of PeerBridge

2.2 Component Overview

Layer	Component	Responsibility
Presentation	Auth & Profile Pages	Register, login, view and edit profile
Presentation	Session Pages	List with filters, details, create/edit form, My Sessions

Layer	Component	Responsibility
Presentation	Quiz Pages	Quiz builder for mentors; timed attempt and result for students
Presentation	Leaderboard Pages	Student and mentor leaderboards, own rank, badges
Presentation	Rating Pages	Rating form and mentor rating history
Presentation	Resource Pages	Search, share, upvote, bookmark resources
Presentation	Shared (Router, AuthContext, Axios)	Route protection by role, token storage, API calls
API	Routes and Controllers	Receive requests, call services, return JSON
API	JWT and Role Middleware	Verify token; allow only the correct role
API	Validation and Error Handler	Check input; give clear error messages and status codes
Service	Auth Service	Register (hash password, set role), login, profile
Service	Session Service	Create, edit, delete, filter, register, list students
Service	Quiz Service	Create quiz, start attempt, autosave, grade on server
Service	Points and Badge Service	Award points once, update total, set badge
Service	Rating Service	Save rating, recompute mentor average
Service	Leaderboard Service	Ranked lists and own rank
Service	Resource Service	Share, search, upvote, bookmark
Service	Points Rules Config	Fixed point values and badge ranges
Data	MongoDB Atlas	8 collections through Mongoose models; automatic backup
External	Meeting links	Google Meet / Zoom / Teams links pasted by the mentor

2.3 Main Data Flow

The sequence below shows the core journey of a junior: login, register for a session, attempt the quiz and rate the mentor.

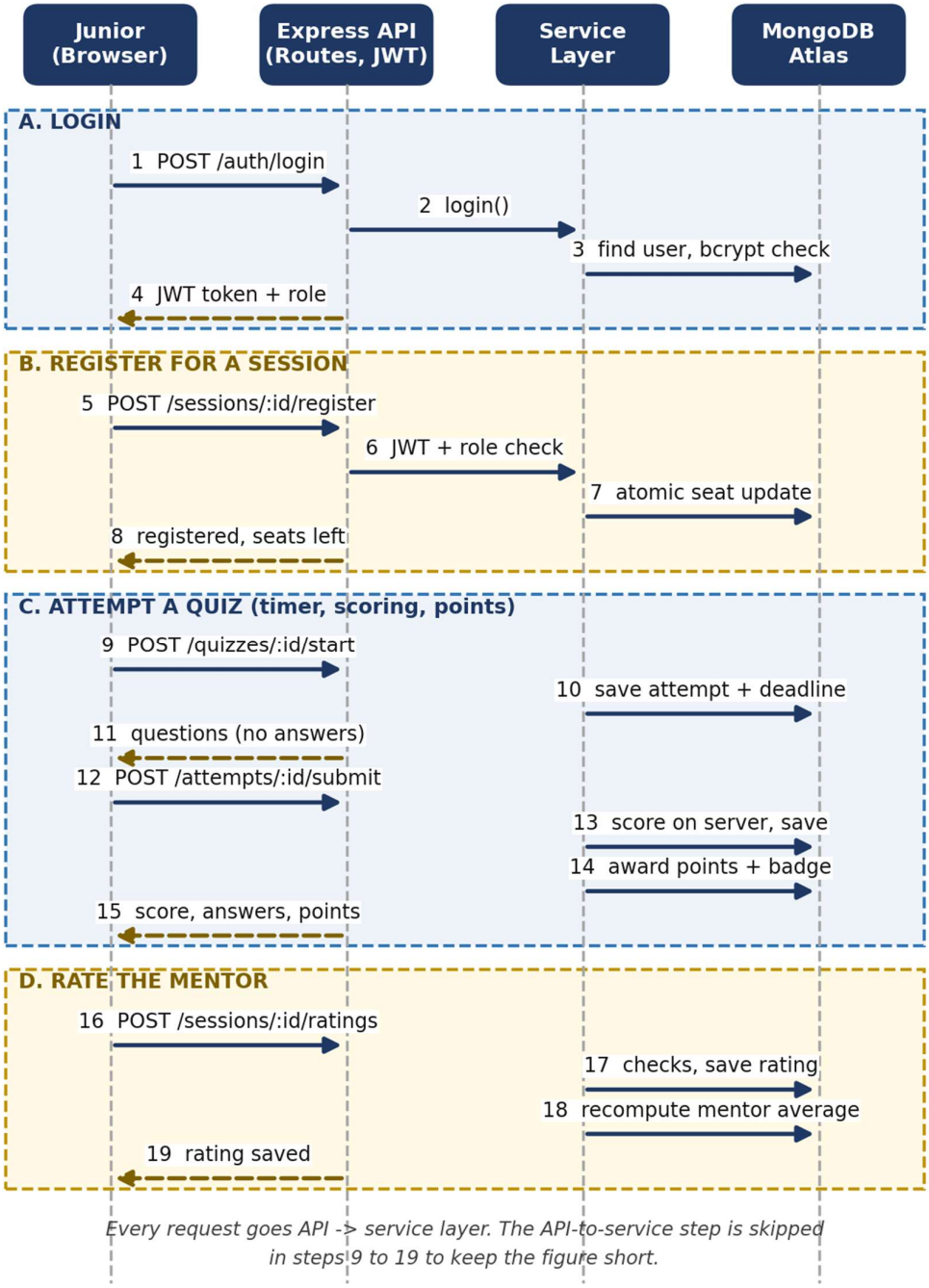


Figure 2: Sequence of the main student journey

2.4 Data Design (Overview)

Eight collections. Details are in the LLD.

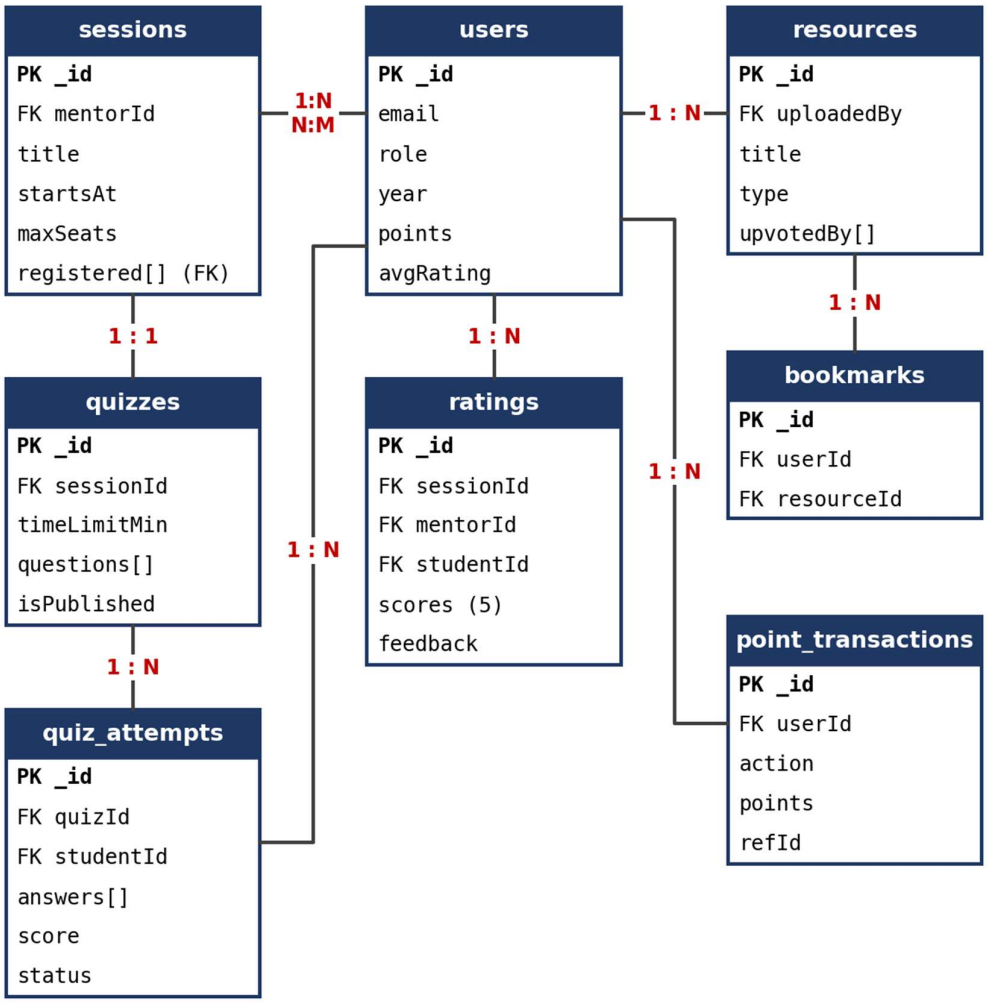


Figure 3: Collection relationship diagram

Only key fields are shown. Full fields are in Section 3.3. Other links: ratings refer to sessions and users; quiz_attempts refer to users (student); bookmarks refer to users.

2.5 External Interfaces

Interface	Direction	Details
Student browser	In / Out	HTTPS REST calls using JSON; JWT in the Authorization header
MongoDB Atlas	Out	Mongoose over a secure connection; free tier (M0)
Meeting platforms	None (link only)	The link is saved and shown. No API call is made
Hosting	Deploy	Frontend on Vercel, backend on Render (or localhost, see OI-4)

2.6 Module Interaction

From	To	What happens
Session (register)	Quiz	Only registered students can start the quiz
Session (register)	Rating	Only registered students can rate, after the start time
Quiz (submit)	Leaderboard	Quiz points are added to the student's total
Rating (new)	Leaderboard	Mentor average rating is recomputed and the mentor ranking changes
Resource (share)	Leaderboard	Small points for sharing, within a daily cap
Points total	Profile	Points, badge and rank are shown on the profile

2.7 Requirement to Design Mapping

Requirement	How the design meets it
AUTH-F1, F2, N5 Register, bcrypt, unique email	AuthService.register(); bcrypt hash before saving; unique index on email
AUTH-F3 Role from year	Role calculated in the service; never read from the request body
AUTH-F4, F5 Login and JWT	AuthService.login() returns a signed JWT (24 hours)
AUTH-F6, F7, N2 Protected pages and roles	JWT middleware and requireRole(); ProtectedRoute in React
AUTH-N3 Safe login error	Same message for wrong email or wrong password
AUTH-F8, F9 Profile, points and rank	Profile pages; users.points; LeaderboardService.myRank()
SESS-F1, F2, N3 Create, edit own sessions	SessionForm; service checks mentorId equals the logged-in user
SESS-F3, F4, N2 List and filter	GET /sessions with filters, startsAt in the future, pagination, indexes
SESS-F5 to F8, N1 Registration rules	Atomic update (Section 3.5.2); seats left = maxSeats - registered count
SESS-F9, F10 My Sessions, student list	GET /sessions/mine; GET /sessions/:id/students (owner only)
QUIZ-F1, F2 Create quiz	quizzes.questions[] with options, correctIndex, marks
QUIZ-F3 Registered only	Service checks registeredStudents before starting an attempt
QUIZ-F4, N1 No early answers	correctIndex and explanation are removed from the question response
QUIZ-F5, F6, N2, N4 Server scoring	QuizService grades on submit and returns result immediately
QUIZ-F7 Points	PointsService.award() called from submit
QUIZ-F8, N3 Timed, refresh safe	deadlineAt on the server; autosave; finalizeIfExpired()
LEAD-F1 to F3, N1 Points and ranking	point_transactions plus users.points; sorted query on a points index
LEAD-F4 Mentor leaderboard	Seniors with 3 or more ratings, sorted by avgRating
LEAD-F5 Badge	Badge ranges in the points config
LEAD-F6 Own position	GET /leaderboard/me; own row highlighted in the table
LEAD-N2 Fast load	Index on points; page size 20; tested for 500 students
LEAD-N3 Fixed values	One config file, no random values
RATE-F1, N3 Rate after attending	Registered check and session start time check
RATE-F2, F6, F7 Five scores, feedback	ratings fields; MentorRatingsPage
RATE-F3, N1 Once per session	Unique index (sessionId, studentId)
RATE-F4 No self rating	Rejected if studentId equals mentorId
RATE-F5 Average	RatingService.recomputeAverage() after every new rating
RES-F1 to F5 Resources	ResourceService; text index; upvotedBy; bookmarks; uploadedBy and createdAt

Requirement	How the design meets it
RES-N2 Broken links	Link shown as a normal anchor; server never fetches it
SYS-3, SYS-8 Security and backup	bcrypt, JWT, HTTPS; Atlas automatic backup

3. Low Level Design (LLD)

3.1 Frontend Components

Component	Props / State	Behaviour
AuthContext	user, token	Stores login state; login(), logout(); reads token on reload
ApiClient (Axios)	base URL	Adds JWT header; on 401 logs out and goes to login
ProtectedRoute	allowedRoles	Blocks pages for logged-out users or the wrong role
Navbar	user role	Links for all main sections; different links for junior and senior
RegisterPage, LoginPage	form fields	Calls auth API; year field decides the role on the server
ProfilePage	profile, points, badge, rank	View and edit bio, skills, links, photo
JuniorDashboard, SeniorDashboard	summary data	Upcoming sessions, points, quick links; senior also sees created sessions
SessionListPage	sessions, filters, page	Search and filter; empty-state message when nothing matches
SessionCard	session	Title, mentor, date, difficulty, seats left
SessionDetailPage	session	Details and Register button (disabled when full or already registered)
SessionForm	session or empty	Create and edit; validates meeting link and seats
MySessionsPage	registered, created	Two tabs; mentor can open the student list of each session
QuizBuilderPage	questions[]	Add questions, options, correct option, marks, time limit; publish
QuizAttemptPage	attempt, questions	Shows one or all questions; autosaves each answer
QuizTimer	deadlineAt, serverNow	Countdown from the server deadline; auto-submits at 0; safe on refresh
QuizResultPage	result	Score, correct answers, explanations, points earned
LeaderboardPage	tab, rows, myRank	Tabs: Students and Mentors; shows own position
LeaderboardTable, BadgeChip	rows, badge	Ranked rows; own row highlighted; badge label
RatingForm, StarInput	scores, feedback	Five 1-to-5 star inputs and optional text
MentorRatingsPage	mentor, ratings	Average rating and feedback list
ResourceListPage	resources, filters	Search by text; filter by subject and type
ResourceCard	resource	Link, uploader, date, Upvote and Bookmark buttons
ResourceForm, BookmarksPage	form fields, bookmarks	Share a link; list saved resources
Toast, EmptyState	message	Success and error messages; no-data message

3.2 Backend Classes

Class	Main methods	Notes
AuthController / AuthService	register(), login(), getProfile(), updateProfile()	bcrypt hashing; JWT expiry 24 hours
authMiddleware	verifyToken()	Reads the Authorization header; attaches user to the request
roleMiddleware	requireRole(role)	Returns 403 for the wrong role
validators	validate(schema)	Checks body, params and query (express-validator or Joi)
SessionController / SessionService	create(), update(), remove(), list(filters), getOne(), register(), mine(), listStudents()	Owner check on edit, delete and student list
QuizController / QuizService	createQuiz(), updateQuiz(), startAttempt(), saveAnswer(), submitAttempt(), getResult(), finalizeIfExpired()	Grading only on the server
PointsService	award(userId, action, refId), getBadge(points)	Uses the points config; ignores duplicate awards
RatingController / RatingService	create(), recomputeAverage(), listForMentor()	Checks registration, start time, duplicate, self-rating
LeaderboardController / LeaderboardService	students(page), mentors(page), myRank(userId)	Ties share the same rank
ResourceController / ResourceService	create(), search(), toggleUpvote(), toggleBookmark(), listBookmarks()	Validates http/https link
errorHandler	handle(err)	Maps errors to status codes; hides internal details
Mongoose models	User, Session, Quiz, QuizAttempt, Rating, Resource, Bookmark, PointTransaction	Only models access the database

3.3 Database Schema (MongoDB Collections)

users

Field	Type	Constraint
`_id`	`ObjectId`	PK
`name`	`String`	required
`email`	`String`	unique, lowercase, required
`passwordHash`	`String`	required; never returned by the API
`college`	`String`	required
`year`	`Number`	1 to 4
`role`	`String`	junior or senior; set by server from year
`bio, skills[], socialLinks, photoUrl`	`String, [String], Object, String`	optional
`points`	`Number`	default 0
`badge`	`String`	from points ranges
`avgRating, ratingCount`	`Number`	default 0; used for mentor leaderboard
`createdAt`	`Date`	auto

sessions

Field	Type	Constraint
`_id`	`ObjectId`	PK
`title, subject, category`	`String`	required

Field	Type	Constraint
difficulty	String	Beginner, Intermediate or Advanced
startsAt	Date	required; UTC
meetingLink	String	valid http/https URL
mentorId	ObjectId	ref users; must be a senior
maxSeats	Number	at least 1
registeredStudents[]	[ObjectId]	ref users; no duplicates; size never above maxSeats
createdAt	Date	auto

quizzes

Field	Type	Constraint
_id	ObjectId	PK
sessionId	ObjectId	ref sessions; unique (one quiz per session)
title, timeLimitMin	String, Number	time limit at least 1 minute
questions[]	Array	each: text, options[] (2 to 6), correctIndex, marks (default 1), explanation
totalMarks	Number	sum of marks; set on save
isPublished	Boolean	students see it only when true
createdBy	ObjectId	ref users (mentor)

quiz_attempts (UNIQUE quizId + studentId)

Field	Type	Constraint
_id	ObjectId	PK
quizId, sessionId, studentId	ObjectId	refs
startedAt, deadlineAt	Date	server time; deadline = start + time limit
answers[]	Array	each: questionId, selectedIndex
status	String	IN_PROGRESS, SUBMITTED or AUTO_SUBMITTED
score, totalMarks, percent	Number	set on grading
pointsAwarded	Number	from PointsService
submittedAt	Date	set on submit

ratings (UNIQUE sessionId + studentId)

Field	Type	Constraint
_id	ObjectId	PK
sessionId, mentorId, studentId	ObjectId	refs
subjectKnowledge, explanationQuality, communication, usefulness, overall	Number	each 1 to 5
feedback	String	optional, up to 500 characters
createdAt	Date	auto

resources, bookmarks, point_transactions

Collection	Fields
`resources`	_id, title, description, subject, type (PDF, NOTES, VIDEO, GITHUB, OTHER), link, uploadedBy (ref users), upvotedBy[], upvoteCount, createdAt
`bookmarks`	_id, userId, resourceId, createdAt (UNIQUE userId + resourceId)
`point_transactions`	_id, userId, action, points, refType, refId, createdAt (UNIQUE userId + action + refId)

Indexes

Collection	Index	Purpose
users	email (unique)	Login; no duplicate email
users	points (descending)	Student leaderboard
users	role, avgRating (descending), ratingCount	Mentor leaderboard
sessions	startsAt; subject, category, difficulty; mentorId	Upcoming list, filters, My Sessions
quiz_attempts	quizId + studentId (unique)	One attempt per student
ratings	sessionId + studentId (unique); mentorId	One rating per session; mentor history
resources	text on title + description; subject, type	Search and filter
point_transactions	userId + action + refId (unique)	No double award

Points rules (proposed, config file; OI-6)

Action	Points	Rule
SESSION_ATTENDED	+5	Once per session, when the student first starts the quiz or submits the rating
QUIZ_COMPLETE	10 to 20	10 for completing + round(percent / 10) as performance bonus; once per quiz
RATING_GIVEN	+2	Once per session
RESOURCE_SHARED	+3	Maximum 3 rewarded shares per day

Badge	Points range
Newcomer	0 to 49
Learner	50 to 149
Achiever	150 to 299
Champion	300 and above

3.4 API Design

Method	Endpoint	Role	Purpose
POST	`/api/auth/register`	Public	Create account; role from year
POST	`/api/auth/login`	Public	Returns JWT and user
GET / PUT	`/api/users/me`	Logged in	View and edit own profile
GET	`/api/users/:id`	Logged in	Public profile of a mentor or student
GET	`/api/sessions`	Logged in	Upcoming sessions; filters and page
GET	`/api/sessions/:id`	Logged in	Session details
POST	`/api/sessions`	Senior	Create a session

Method	Endpoint	Role	Purpose
PUT / DELETE	`/api/sessions/:id`	Owner mentor	Edit or delete
POST	`/api/sessions/:id/register`	Student (not owner)	Register for a session
GET	`/api/sessions/mine`	Logged in	Registered and created sessions
GET	`/api/sessions/:id/students`	Owner mentor	Registered students
POST	`/api/sessions/:id/quiz`	Owner mentor	Create the quiz
PUT	`/api/quizzes/:id`	Owner mentor	Edit or publish the quiz
POST	`/api/quizzes/:id/start`	Registered student	Start the attempt; returns questions and deadline
PATCH	`/api/attempts/:id/answer`	Attempt owner	Autosave one answer
POST	`/api/attempts/:id/submit`	Attempt owner	Submit and grade
GET	`/api/attempts/:id/result`	Attempt owner	Score, answers, explanations
GET	`/api/leaderboard/students`	Logged in	Ranked students; page
GET	`/api/leaderboard/mentors`	Logged in	Ranked mentors
GET	`/api/leaderboard/me`	Logged in	Own rank and points
POST	`/api/sessions/:id/ratings`	Registered student	Rate the mentor
GET	`/api/mentors/:id/ratings`	Logged in	Average and feedback
GET / POST	`/api/resources`	Logged in	Search and filter; share a resource
POST	`/api/resources/:id/upvote`	Logged in	Toggle upvote
POST	`/api/resources/:id/bookmark`	Logged in	Toggle bookmark
GET	`/api/resources/bookmarks`	Logged in	Own bookmarks

Sample: register (request) and login (response)

```
POST /api/auth/register
{ "name": "Ravi", "email": "ravi@college.edu", "password": "******",
  "college": "ABC College", "year": 2 }          // role = junior (set by server)

POST /api/auth/login  ->
{ "token": "<JWT>", "user": { "id": "66f1...", "name": "Ravi", "role": "junior", "points": 0 } }
```

Sample: start quiz attempt (response, no answers sent)

```
{
  "attemptId": "6701...", "serverNow": "2026-10-10T10:00:00Z",
  "deadlineAt": "2026-10-10T10:15:00Z",
  "questions": [ { "id": "q1", "text": "What is a stack?", "options": ["...", "...", "..."], "marks":
2 } ]
}
// correctIndex and explanation are sent only after submit
```

Sample: submit result (response)

```
{ "score": 8, "totalMarks": 10, "percent": 80, "pointsAwarded": 18,
  "badge": "Learner", "review": [ { "questionId": "q1", "correctIndex": 2, "explanation": "..." } ] }
```

3.5 Key Algorithms

3.5.1 Role from Year

```
role = (year <= 2) ? 'junior' : 'senior'      // year must be 1..4; body.role is ignored
```

3.5.2 Seat Registration (atomic)

```
register(userId, sessionId):
  updated = Session.findOneAndUpdate(
    { _id: sessionId,
      mentorId: { $ne: userId },                        // no self registration
      registeredStudents: { $ne: userId },              // no duplicate
      startsAt: { $gt: now },                           // session not started
      $expr: { $lt: [ { $size: '$registeredStudents' }, '$maxSeats' ] } },   // seat free
    { $addToSet: { registeredStudents: userId } }, { new: true })
  if updated: return seatsLeft = maxSeats - updated.registeredStudents.length
  else: read the session and return the reason: 404, 400 own session, 409 already registered, 409 full
```

Because the checks and the update happen in one database operation, two students cannot take the last seat together.

3.5.3 Quiz Attempt and Scoring

```
startAttempt(userId, quizId):
  require quiz.isPublished and userId in session.registeredStudents
  attempt = QuizAttempt.create({ quizId, studentId, startedAt: now, deadlineAt: now + timeLimit })
     // unique index (quizId, studentId): second start returns the existing attempt
  return questions WITHOUT correctIndex and explanation, plus deadlineAt and serverNow

submitAttempt(attemptId, userId):
  finalizeIfExpired(attempt)            // status AUTO_SUBMITTED if now > deadlineAt + 5 s
  score = 0
  for each question q: if answer(q) == q.correctIndex: score += q.marks
  percent = score / totalMarks * 100
  save score, percent, status, submittedAt
  points = 10 + round(percent / 10)
  PointsService.award(userId, 'QUIZ_COMPLETE', quizId, points)
```

3.5.4 Award Points (no double award)

```
award(userId, action, refId, points):
  try   PointTransaction.create({ userId, action, refId, points })   // unique index
  catch duplicate key: return                                          // already awarded
  user = User.findByIdAndUpdate(userId, { $inc: { points } }, { new: true })
  User.updateOne({ _id: userId }, { badge: getBadge(user.points) })
```

3.5.5 Rating and Mentor Average

```
createRating(studentId, sessionId, scores, feedback):
  require studentId in session.registeredStudents and now >= session.startsAt
  require studentId != session.mentorId
  Rating.create(...)                        // unique index blocks a second rating
  agg = Rating.aggregate([ { $match: { mentorId } },
          { $group: { _id: null, avg: { $avg: '$overall' }, n: { $sum: 1 } } } ])
  User.updateOne({ _id: mentorId }, { avgRating: round(agg.avg, 2), ratingCount: agg.n })
  PointsService.award(studentId, 'RATING_GIVEN', sessionId, 2)
```

3.5.6 Leaderboards and Own Rank

```
students(page): User.find().sort({ points: -1, createdAt: 1 }).skip(...).limit(20)
mentors(page):  User.find({ role: 'senior', ratingCount: { $gte: 3 } })
                    .sort({ avgRating: -1, ratingCount: -1 }).skip(...).limit(20)
myRank(userId): count( User where points > me.points ) + 1      // equal points share a rank
```

3.5.7 Worked Example

Case	Calculation	Result
Quiz: 10 marks in total, student scores 8	percent = 80; points = 10 + round(80 / 10)	18 points
Total points 18 + 5 (attended) + 2 (rating)	18 + 5 + 2	25 points, badge Newcomer
Mentor ratings (overall): 5, 4, 5	(5 + 4 + 5) / 3	4.67, appears on the mentor leaderboard (3 ratings)
Session with 30 seats, 30 registered	registered size is not less than maxSeats	31st student gets 409 Full

3.6 Quiz Timer and State Logic

State	Meaning	Next state
IN_PROGRESS	Attempt started; answers autosaved	SUBMITTED, AUTO_SUBMITTED
SUBMITTED	Student submitted before the deadline; graded	none
AUTO_SUBMITTED	Deadline passed; graded with the answers saved so far	none

- deadlineAt is saved once at start. A page refresh reads the same deadline, so the timer never resets (QUIZ-N3)
- The browser shows a countdown from deadlineAt minus serverNow; changing the device clock has no effect
- The browser submits automatically at 0. If it does not, the server closes the attempt the next time it is accessed (finalizeIfExpired)
- An answer that arrives after the deadline plus 5 seconds is ignored
- A student cannot start a second attempt; the same attempt is returned

Session status (not stored, calculated)

Status	Condition	What the student can do
Upcoming	startsAt is in the future	Register; appears in the list
Started or past	startsAt has passed	Attempt quiz (if published); rate the mentor

3.7 Validation and Error Handling

Case	Check	Response
Token missing or expired	JWT middleware	401 Unauthorized
Wrong role (e.g. junior creates session)	requireRole()	403 Forbidden
Duplicate email	Unique index	409 with a clear message
Wrong email or password	bcrypt compare	401: Invalid email or password (same message)
Year outside 1 to 4	Body validation	400
Edit or delete another mentor's session	Owner check	403
Register twice, own session, or full	Atomic update result	409 or 400 with the reason
Start quiz without registration	Registered check	403
Answer after the deadline	deadlineAt + 5 s	Ignored; attempt closed
Rate twice or rate own session	Unique index and check	409 or 400
Rating outside 1 to 5	Range check	400
Invalid resource link	http/https URL check	400
Database or server failure	Error handler	500 with a generic message; details only in server logs

3.8 Security

- Passwords hashed with bcrypt; never sent back to the browser
- JWT in the Authorization header; token expires in 24 hours
- Role and ownership checked on the server for every protected action
- Correct answers stay on the server until the quiz is submitted
- Input validated; Mongoose queries used safely to block NoSQL injection (no raw objects from the request in filters)

- CORS allows only the frontend URL; rate limit on login
- Only name, email, college and year are collected (SYS-2); secrets kept in environment variables

3.9 Test Cases

ID	Test	Expected result
TC1	Register with year 2	Role is junior
TC2	Register with year 4 and role=junior in the body	Role is senior; body role ignored
TC3	Register with an existing email	409 error
TC4	Login with a wrong password	Generic error message
TC5	Junior opens the create session page	Blocked (403)
TC6	Mentor edits another mentor's session	403
TC7	Register for a session twice	Second request is rejected
TC8	Two students take the last seat together	One succeeds; one gets Full
TC9	Mentor registers for own session	Rejected
TC10	Unregistered student starts the quiz	403
TC11	Check the quiz response in the browser	No correctIndex or explanation
TC12	Refresh during the quiz	Same deadline; timer continues
TC13	Let the timer reach 0	Auto-submitted and graded
TC14	Submit the same quiz again	Same result; points not added twice
TC15	Rate a session twice	Second rating rejected
TC16	Rate before the session start time	Rejected
TC17	Add a rating	Mentor average changes correctly
TC18	Student with 25 points opens the leaderboard	Own row highlighted with correct rank
TC19	Mentor with 2 ratings	Not on the mentor leaderboard
TC20	Share a resource with a broken link	Page still loads; link opens as given
TC21	Upvote the same resource twice	Second click removes the upvote (toggle)